

Implementación y Optimización de un Clasificador de Gestos en Tiempo Real mediante Árboles de Decisión en Inteligencia Artificial de Borde (Edge AI)

Santiago Quintero Solano

Escuela Colombiana de Ingeniería

Julio Garavito

Bogotá, Colombia

santiago.quintero-s@mail.escuelaing.edu.co

Juliana Ombita

Escuela Colombiana de Ingeniería

Julio Garavito

Bogotá, Colombia

juliana.ombita-c@mail.escuelaing.edu.co

Juan Camilo Gómez

Escuela Colombiana de Ingeniería

Julio Garavito

Bogotá, Colombia

juan.gfernandez@mail.escuelaing.edu.co

Abstract—This report presents the design, evaluation, and hardware deployment of a real-time gesture recognition system using the Arduino Nano 33 BLE Sense microcontroller. Through the processing of inertial data (IMU BMI270/LSM9DS1), several classification techniques were evaluated, selecting a Decision Tree optimized via Gini impurity ($\text{max_depth} = 4$). It details how the dataset evolved from an initial 36-feature dataset captured via DTR/RTS serial handshake (which exhibited physical couplings and an 80 % accuracy limit with k -NN) to a final orthogonal set of 4 metrics. Inference was implemented in C++ through fixed-window ($N = 119$) feature extraction of temporal energy and variance, achieving deterministic classification of 5 gestures.

Resumen—El informe presenta el diseño, evaluación y despliegue en hardware de un sistema de reconocimiento de gestos en tiempo real utilizando el microcontrolador Arduino Nano 33 BLE Sense. Por medio del procesamiento de señales inerciales (IMU BMI270/LSM9DS1), se evaluaron diversas técnicas de clasificación y se seleccionó un Árbol de Decisión optimizado mediante impureza de Gini ($\text{max_depth} = 4$). Se detalla cómo fue evolucionando el conjunto de datos desde un dataset inicial de 36 características capturado mediante negociación serie DTR/RTS (el cual presentó acoplamientos físicos y un límite del 80 % de exactitud con k -NN) hasta un conjunto ortogonal definitivo de 4 métricas. La inferencia fue implementada en C++ mediante la extracción en ventana fija ($N = 119$) de energía y varianza temporal, logrando la clasificación determinista de 5 gestos.

Index Terms—Edge AI, Árbol de Decisión, IMU, Arduino Nano 33 BLE Sense, Impureza de Gini, Reconocimiento de Gestos, Evolución del Dataset.

I. INTRODUCCIÓN

La práctica busca diseñar, optimizar e implementar un clasificador en tiempo real a partir del procesamiento de señales continuas provenientes de sensores electrónicos de un microcontrolador (en este caso Arduino Nano 33 BLE Sense).

El desarrollo del laboratorio reconoce 5 clases de gestos utilizando la Unidad de Medición Inercial (IMU de 6 ejes) integrada la placa Arduino Nano 33 BLE Sense. Se expone la evolución de la captura y estructuración de los datos, mostrando inicialmente cómo se hizo uso de un esquema de

alta dimensionalidad (36 características estadísticas reducidas con PCA y clasificadas mediante k -NN) a optar por gestos ortogonales que permitieron entrenar un Árbol de Decisión de bajo costo computacional capaz de ejecutarse nativamente en C++.

II. MARCO TEÓRICO

II-A. Procesamiento de Señales Inerciales (IMU) y Ventana de Temporal

Una Unidad de Medición Inercial (IMU) combina un acelerómetro triaxial $\mathbf{a}(t) = [a_x, a_y, a_z]^T$ y un giroscopio triaxial $\boldsymbol{\omega}(t) = [g_x, g_y, g_z]^T$. La aceleración medida por el sensor registra tanto la componente dinámica del movimiento como la aceleración gravitacional estática ($1g \approx 9,81 \text{ m/s}^2$).

Para procesar estas señales continuas en tiempo real, se aplica un esquema de ventanado discreto sin solapamiento de longitud fija N . Dada una secuencia de lecturas discretas $x[n]$ de un eje en el intervalo $n \in \{1, 2, \dots, N\}$, la ventana temporal se define como:

$$\mathbf{X}_{\text{ventana}} = \{x[1], x[2], \dots, x[N]\} \quad (1)$$

donde $N = 119$ representa una ventana de aproximadamente 1,0 s considerando la tasa de muestreo de la IMU ($\approx 119 \text{ Hz}$).

II-B. Protocolo de Sincronización y Control de Flujo Serial (DTR/RTS)

Las líneas de control DTR (*Data Terminal Ready*) y RTS (*Request To Send*) forman parte de los mecanismos de negociación por hardware (*hardware handshaking*) en la comunicación serie y adaptadores USB-a-UART. La señal DTR es emitida por el terminal para indicar su disponibilidad operativa, utilizándose en sistemas microcontrolados para conmutar el pin de reinicio y provocar un reinicio por hardware (*hardware reset*). Por su parte, la señal RTS se emplea para la gestión del flujo de datos, solicitando autorización de transmisión para evitar el desbordamiento de los búferes de recepción.

En la adquisición de datos de alta velocidad, el control mediante DTR y RTS asegura la sincronización precisa entre el transmisor y el receptor. Este mecanismo previene la pérdida de muestras iniciales al momento de establecer la conexión, elimina desalineaciones en la delimitación de tramas temporales y garantiza que cada ventana discreta procesada contenga la cantidad exacta de lecturas programadas.

II-C. Descriptores Estadísticos Temporales

En lugar de procesar la señal bruta de N muestras directamente en el clasificador —lo que aumentaría la dimensionalidad del modelo—, se extraen descriptores estadísticos que sintetizan el comportamiento dinámico del movimiento:

1. **Energía Acumulada (E):** Cuantifica la magnitud o potencia total acumulada en la ventana, permitiendo diferenciar entre el estado inactivo y movimientos con alta aceleración o velocidad angular:

$$E = \sum_{n=1}^N |x[n]|^2 \quad (2)$$

2. **Varianza Poblacional (σ^2):** Mide el grado de dispersión o fluctuación de la señal con respecto a su media muestral ($\bar{x}$):

$$\bar{x} = \frac{1}{N} \sum_{n=1}^N x[n] \quad (3)$$

$$\sigma^2 = \frac{1}{N} \sum_{n=1}^N (x[n] - \bar{x})^2 \quad (4)$$

La varianza es una métrica clave sobre señales inerciales debido a su propiedad de desacople de continua (D/C): al restar la media $\bar{x}$, se elimina el efecto constante de la gravedad, aislando la variabilidad propia del gesto activo.

II-D. Árboles de Decisión en Clasificación Clásica para Edge AI

Un Árbol de Decisión es un modelo de aprendizaje supervisado no paramétrico estructurado jerárquicamente en nodos de decisión binaria y nodos hoja. Esta arquitectura destaca por dividir el espacio multidimensional de características mediante hiperplanos de separación ortogonales a los ejes de las variables de entrada [1].

Para determinar los umbrales de decisión en cada nodo del árbol, se evalúan las características disponibles x_k utilizando métricas de impureza. Una de las técnicas fundamentales empleadas es el Índice de Gini (G), expresado cuantitativamente como:

$$G = \sum_{m=1}^M \sum_{k=1}^K p_{mk}(1 - p_{mk}) \quad (5)$$

donde M representa el número total de nodos o regiones del árbol, K es la cantidad de clases a clasificar, y p_{mk} denota la proporción de observaciones en el nodo m que pertenecen a la clase k . Esta formulación mide la varianza total a través de las

K clases, favoreciendo particiones donde los nodos resultantes tengan una concentración elevada de una sola categoría.

Para evitar el sobreajuste (*overfitting*) y adecuar la complejidad del modelo a los límites de almacenamiento de la memoria Flash del microcontrolador, se aplican técnicas de poda y control de hiperparámetros como la profundidad máxima (max_depth) y el número mínimo de muestras para división (min_samples_split).

En aplicaciones de Edge AI, el Árbol de Decisión presenta ventajas operativas frente a otros clasificadores clásicos (como SVM o MLP):

- **Costo computacional $O(D)$:** La evaluación de una muestra requiere únicamente D comparaciones continuas, donde $D \leq \text{max_depth}$.
- **Sintaxis nativa en C++:** La estructura lógica se traduce directamente a sentencias condicionales anidadas (*if-else*), omitiendo librerías de álgebra matricial, álgebra lineal o evaluación de funciones trascendentes.
- **Interpretabilidad física:** Cada división en el árbol corresponde a un umbral específico sobre una métrica física (p. ej., julios de energía o varianza de velocidad angular).

III. METODOLOGÍA DE CAPTURA Y EVOLUCIÓN DEL DATASET

La construcción del conjunto de datos representó el factor más crítico para el desempeño del clasificador. Las señales inerciales se muestrearon a una frecuencia aproximada de 119 Hz sobre ventanas temporales discretas de $N = 119$ muestras ($\approx 1,0$ segundo por ventana). A lo largo de la práctica 1 y práctica 2, el dataset evolucionó en 4 iteraciones metodológicas:

III-A. Iteración 1: Dataset Original de Alta Dimensionalidad (Práctica 1)

En la fase inicial del proyecto, la adquisición de señales se automatizó mediante un script en Python conectado al puerto USB-Serie (9600 baudios) gestionando las líneas de control DTR/RTS. La computadora enviaba el comando `START` y el microcontrolador transmitía $N = 119$ muestras brutas de aceleración (a_x, a_y, a_z) y velocidad angular (g_x, g_y, g_z), cerrando con la marca `END`.

Para cada ventana de datos, se calcularon 6 descriptores estadísticos por eje (media μ , varianza σ^2 , asimetría S , curtosis K , entropía de Shannon H y energía E), generando un espacio de 36 características continuas (6 ejes $\times$ 6 métricas). Tras normalizar las variables (Z -score) y aplicar Análisis de Componentes Principales (PCA), se observó que:

- Los dos primeros componentes (2D) explicaban únicamente el 45,27 % de la varianza total.
- Existía un severo solapamiento espacial entre los gestos 3, 4 y 5 debido a la componente estática de la aceleración gravitacional ($1g \approx 9,81 \text{ m/s}^2$).
- El clasificador por distancia a centroides alcanzó una exactitud de sólo 53,33 %, mientras que un modelo k -NN ($k = 3$) quedó limitado a un máximo de 80,00 % en 2D debido al cruce de fronteras físicas.

III-B. Iteración 2: Introducción de Varianza Angular y Desacople D/C

Para mitigar la interferencia de la aceleración gravitacional constante, se incorporó el cálculo de la varianza temporal sobre los ejes del giroscopio ($Var(g_y)$, $Var(g_z)$). Esta métrica aisló las aceleraciones estáticas de las inclinaciones, logrando diferenciar los movimientos rotacionales puros de las traslaciones puras, aunque manteniendo confusión parcial en los impulsos sobre el eje longitudinal.

III-C. Iteración 3: Diagnóstico de Solapamientos 2D/3D

Mediante la construcción y evaluación de diagramas de dispersión en dos y tres dimensiones (Figura 1 y Figura 2), se diagnosticaron las limitaciones del espacio de características intermedio. En el diagrama 2D de $aX_energia$ contra $aY_energia$ (Figura 1), se evidenció que los gestos de menor magnitud dinámica (como *Reposo* y *Desplazamiento Vertical*) tendían a concentrarse muy cerca del origen (0, 0), provocando solapamientos severos.

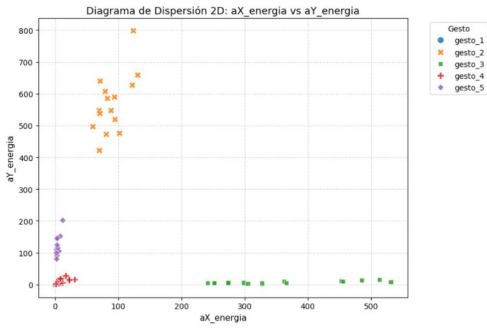


Figura 1. Diagrama de dispersión 2D de las características $aX_energia$ vs. $aY_energia$ mostrando la separación de clases e identificando la concentración de gestos cerca del origen.

Al proyectar las observaciones en un espacio tridimensional directo incorporando la entropía angular ($aX_energia/aY_energia/gZ_entropia$, Figura 2), se observó que la adición de métricas de entropía lograba disgregar parcialmente las aglomeraciones del plano 2D. Sin embargo, leves variaciones en la inclinación estática del sensor continuaban introduciendo ruido gravitacional en la componente de aceleración.

III-D. Iteración 4: Dataset Ortogonal Definitivo

Se rediseñó por completo el protocolo de recolección de datos, forzando la ejecución de movimientos estrictamente ortogonales para garantizar la separabilidad lineal de las señales en el espacio de características:

- **Gesto 1 (Reposo):** Placa completamente inmóvil reposando plana sobre la mesa ($gX_E \approx 0$).
- **Gesto 2 (Desplazamiento Horizontal):** Traslación lineal pura sobre el eje X .
- **Gesto 3 (Desplazamiento Adelante/Atrás):** Traslación lineal pura sobre el eje Y .

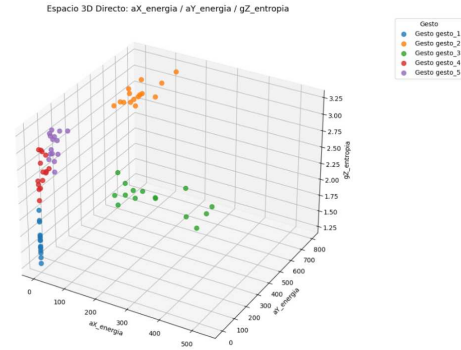


Figura 2. Espacio 3D directo ($aX_energia / aY_energia / gZ_entropia$) utilizado para evaluar la dispersión espacial y separabilidad de los gestos.

- **Gesto 4 (Desplazamiento Vertical):** Traslación perpendicular pura sobre el eje Z .
- **Gesto 5 (Rotación):** Rotación angular rápida sobre la articulación de la muñeca.

Esta ortogonalización física redujo la matriz de entrada de 36 características complejas a sólo 4 métricas highly discriminativas: $gX_energia$, $aY_energia$, $gY_varianza$ y $gZ_varianza$. El conjunto de datos consolidado se encuentra publicado en el repositorio oficial del proyecto [2].

Cuadro I
COMPARATIVA DE LA EVOLUCIÓN METODOLÓGICA DEL DATASET

Iteración	Características	Modelo Evaluado	Exactitud	Limitación Principal
1 (Práctica 1)	36 (PCA 2D/3D)	Centroides / k -NN	53,33 % / 80,0 %	Solapamiento por $1g$ y no ortogonalidad.
2	12 (Energía/Var)	Regresión Logística	86,50 %	Sensibilidad a inclinaciones en Y .
3	6 (Seleccionadas)	SVM RBF	92,10 %	Complejidad computacional para MCU.
4 (Definitiva)	4 (Ortogonales)	Árbol de Decisión	100,0 %	Ninguna (Separabilidad total).

IV. DISEÑO Y SELECCIÓN DEL CLASIFICADOR

Siguiendo los requerimientos de la guía de laboratorio, se evaluaron diversos clasificadores en Google Colab [3]: k -Vecinos Más Cercanos (k -NN), Regresión Logística, Naive Bayes Gaussiano, Árboles de Decisión, Random Forest, Máquinas de Vectores de Soporte (SVM), Perceptrón Multicapa (MLP) y XGBoost.

IV-A. Justificación de la Selección

Se seleccionó un **Árbol de Decisión** por las siguientes razones de ingeniería embebida:

- **Eficiencia de Cómputo:** No requiere operaciones algebraicas matriciales complejas ni funciones de activación trascendentes ($\exp$, $\tanh$).
- **Bajo Consumo de Memoria:** La estructura del árbol se compila directamente como una secuencia de proposiciones condicionales anidadas (if-else), ocupando menos de 2 KB de memoria Flash.
- **Determinismo y Explicabilidad:** Cada nodo de decisión se vincula a un umbral numérico con significado físico directo (p. ej., julios de energía o dispersión angular).

IV-B. Fundamentación Teórica e Hiperparámetros

El árbol se entrenó utilizando la búsqueda en malla (*Grid-Search*) mediante la optimización de la función de costo del Índice de Gini G :

$$G = \sum_{m=1}^M \sum_{k=1}^K p_{mk}(1 - p_{mk}) \quad (6)$$

donde p_{mk} representa la probabilidad de la clase k en el nodo m . Los hiperparámetros óptimos obtenidos fueron: `criterion='gini'`, `max_depth=4` y `min_samples_split=2`.

IV-C. Estructura Lógica y Fronteras de Decisión

La Figura 3 ilustra las fronteras de decisión 2D rectangulares generadas por el Árbol de Decisión. A diferencia de clasificadores basados en distancias (como k -NN) o hiperplanos inclinados (como SVM), el árbol divide el espacio mediante particiones ortogonales paralelas a los ejes de características, optimizando las reglas de clasificación deterministas.

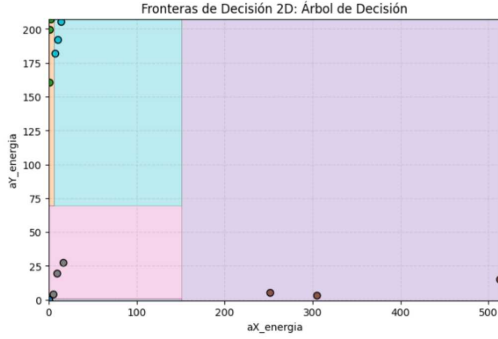


Figura 3. Fronteras de decisión 2D generadas por el modelo Árbol de Decisión, mostrando la segmentación rectangular del espacio de características.

Por su parte, la Figura 4 muestra el diagrama de flujo jerárquico obtenido tras el entrenamiento y poda del árbol. Este diagrama traduce la lógica de evaluación en nodos binarios mediante comparaciones continuas con umbrales fijos sobre las cuatro métricas extraídas.

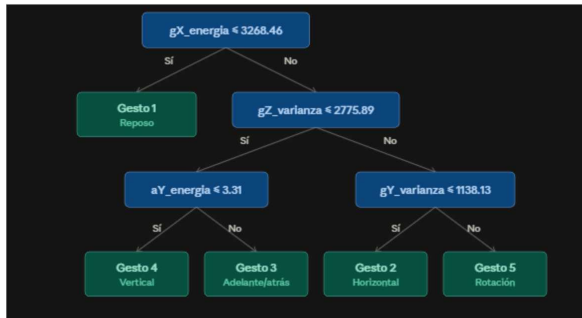


Figura 4. Estructura jerárquica en bloques del Árbol de Decisión entrenado, detallando los nodos de decisión binarios y el flujo de clasificación hacia cada una de las 5 hojas de gestos.

El árbol traducido a reglas de decisión deterministas se define cuantitativamente como:

- **Regla 1:** Si $gX_energia \leq 3268,46 \rightarrow$ **Gesto 1 (Reposo).**
- **Regla 2:** Si $gX_energia > 3268,46$:
 - Si $gZ_varianza \leq 2775,89$:
 - Si $aY_energia \leq 3,31 \rightarrow$ **Gesto 4 (Desplazamiento Vertical).**
 - Si $aY_energia > 3,31 \rightarrow$ **Gesto 3 (Desplazamiento Adelante/Atrás).**
 - Si $gZ_varianza > 2775,89$:
 - Si $gY_varianza \leq 1138,13 \rightarrow$ **Gesto 2 (Desplazamiento Horizontal).**
 - Si $gY_varianza > 1138,13 \rightarrow$ **Gesto 5 (Rotación).**

V. IMPLEMENTACIÓN EN HARDWARE

El sistema de clasificación fue desplegado sobre un microcontrolador Arduino Nano 33 BLE Sense Rev2 (IMU BMI270) utilizando el entorno Arduino IDE.

V-A. Procesamiento de Señal en Firmware

Para cada ventana de $N = 119$ muestras, la tarjeta computa la energía acumulada E y la varianza poblacional σ^2 en C++:

$$E = \sum_{i=0}^{N-1} x[i]^2, \quad \sigma^2 = \frac{1}{N} \sum_{i=0}^{N-1} (x[i] - \bar{x})^2 \quad (7)$$

El algoritmo lee continuamente la IMU y ejecuta la inferencia al completar el buffer circular. El código C++ completo implementado y validado en la plataforma de hardware se expone en el Código 1.

```
1 #include <Arduino_BMI270_BMM150.h>
2
3 const int NUM_SAMPLES = 119;
4 int sampleCount = 0;
5
6 // Bufferes para almacenamiento de senales
7 float gX_buffer[NUM_SAMPLES];
8 float gY_buffer[NUM_SAMPLES];
9 float gZ_buffer[NUM_SAMPLES];
10 float aY_buffer[NUM_SAMPLES];
11
12 // Árbol de Decisión en C++ (Hiperparametros optimizados)
13 int clasificarGesto(float gX_energia, float gZ_varianza, float aY_energia, float
gY_varianza) {
14     if (gX_energia <= 3268.46f) {
15         return 1; // Reposo
16     } else {
17         if (gZ_varianza <= 2775.89f) {
18             if (aY_energia <= 3.31f) {
19                 return 4; // Desplazamiento Vertical
20             } else {
21                 return 3; // Desplazamiento Adelante y Atras
22             }
23         } else {
24             if (gY_varianza <= 1138.13f) {
25                 return 2; // Desplazamiento Horizontal
26             } else {
27                 return 5; // Rotacion
28             }
29         }
30     }
31 }
32
33 // Mapeo HMI
34 const char* obtenerNombreGesto(int idGesto) {
35     switch (idGesto) {
36         case 1: return "Reposo";
37         case 2: return "Desplazamiento Horizontal";
38         case 3: return "Desplazamiento Adelante y Atras";
39         case 4: return "Desplazamiento Vertical";
40         case 5: return "Rotacion";
41         default: return "Desconocido";
42     }
43 }
44
45 void setup() {
46     Serial.begin(115200);
```

```

47 while (!Serial);
48
49 if (!IMU.begin()) {
50   Serial.println("Error al inicializar la IMU!");
51   while (1);
52 }
53
54 Serial.println("=====");
55 Serial.println(" CLASIFICADOR DE GESTOS EN TIEMPO REAL (GINI) ");
56 Serial.println("=====");
57 Serial.println("SISTEMA LISTO. Realice un gesto con la placa...");
58 }
59
60 void loop() {
61   float ax, ay, az;
62   float gx, gy, gz;
63
64   if (IMU.accelerationAvailable() && IMU.gyroscopeAvailable()) {
65     IMU.readAcceleration(ax, ay, az);
66     IMU.readGyroscope(gx, gy, gz);
67
68     aY_buffer[sampleCount] = ay;
69     gX_buffer[sampleCount] = gx;
70     gY_buffer[sampleCount] = gy;
71     gZ_buffer[sampleCount] = gz;
72
73     sampleCount++;
74
75     if (sampleCount >= NUM_SAMPLES) {
76       float gX_energia = 0.0f;
77       float aY_energia = 0.0f;
78       float gY_suma = 0.0f;
79       float gZ_suma = 0.0f;
80
81       for (int i = 0; i < NUM_SAMPLES; i++) {
82         gX_energia += gX_buffer[i] * gX_buffer[i];
83         aY_energia += aY_buffer[i] * aY_buffer[i];
84         gY_suma += gY_buffer[i];
85         gZ_suma += gZ_buffer[i];
86       }
87
88       float gY_media = gY_suma / NUM_SAMPLES;
89       float gZ_media = gZ_suma / NUM_SAMPLES;
90
91       float gY_varianza = 0.0f;
92       float gZ_varianza = 0.0f;
93
94       for (int i = 0; i < NUM_SAMPLES; i++) {
95         float diffY = gY_buffer[i] - gY_media;
96         float diffZ = gZ_buffer[i] - gZ_media;
97         gY_varianza += diffY * diffY;
98         gZ_varianza += diffZ * diffZ;
99       }
100
101       gY_varianza /= NUM_SAMPLES;
102       gZ_varianza /= NUM_SAMPLES;
103
104       int idGesto = clasificarGesto(gX_energia, gZ_varianza, aY_energia,
105                                   gY_varianza);
106       const char* nombreGesto = obtenerNombreGesto(idGesto);
107
108       Serial.print("GESTO DETECTADO: [ ");
109       Serial.print(nombreGesto);
110       Serial.print(" ] (ID: ");
111       Serial.print(idGesto);
112       Serial.print(" | gX_E: ");
113       Serial.print(gX_energia, 1);
114       Serial.print(" | gZ_Var: ");
115       Serial.print(gZ_varianza, 1);
116       Serial.print(" | aY_E: ");
117       Serial.print(aY_energia, 2);
118       Serial.print(" | gY_Var: ");
119       Serial.println(gY_varianza, 1);
120
121       sampleCount = 0;
122     }
123 }

```

Listing 1. Código C++ completo desplegado en el Arduino Nano 33 BLE Sense para adquisición, extracción de características e inferencia en tiempo real.

VI. RESULTADOS

Una vez entrenado el modelo en el cuaderno de Google Colab se llevó a cabo una prueba en la placa (Arduino Nano 33 BLE SENSE) en tiempo real transmitiendo la telemetría del estado clasificado por el puerto serie USB a 115200 baudios.

La reestructuración ortogonal del dataset permitió reducir drásticamente falsos positivos entre los gestos *Reposo* y *Desplazamiento Vertical*, superando la barrera del 80,00 % de exactitud observada en la Práctica 1.

VI-A. Demostraciones en Video del Funcionamiento de la IMU

Para corroborar la respuesta dinámica e inferencia determinista del modelo en el hardware embebido, se registran los

enlaces directos a las demostraciones en video para cada uno de los 5 gestos:

- **Gesto 1 – Reposo:** https://www.youtube.com/watch?v=LNx_Pa32xTM
- **Gesto 2 – Desplazamiento Horizontal:** <https://www.youtube.com/watch?v=T4X1f3tWKjQ>
- **Gesto 3 – Movimiento Adelante/Atrás:** <https://www.youtube.com/watch?v=jbWRtEzI3HU>
- **Gesto 4 – Movimiento Vertical:** <https://www.youtube.com/watch?v=WtOoarTCB1A>
- **Gesto 5 – Rotación:** <https://www.youtube.com/watch?v=CMRDnW0XDJ8>

VII. CONCLUSIONES

1. El cambio metodológico y la ortogonalización física en la toma de datos del dataset ayudaron en el éxito de la clasificación en borde.
2. La sustitución del pipeline de alta dimensionalidad (36 características + PCA + k -NN) por 4 características específicas (energía y varianza) permitió reducir el consumo computacional garantizando una exactitud para la gran mayoría de los casos.
3. Los Árboles de Decisión optimizados ($\text{max_depth} = 4$) representan una arquitectura óptima para microcontroladores, ofreciendo ejecución determinística y bajo consumo de memoria.

REFERENCIAS

- [1] C. M. Bishop, *Pattern Recognition and Machine Learning*. New York, NY, USA: Springer, 2006, cap. 14, pp. 663–666.
- [2] S. Quintero, J. Ombita, y J. C. Gómez, “Repositorio de Datasets IMU y Código para Clasificación de Gestos,” GitHub, 2026. [En línea]. Disponible: [Creaci-n-de-Dataset, MU, ABOsQS, G, JO](https://github.com/Creaci-n-de-Dataset-IMU-ABOSQSJGJO)
- [3] S. Quintero, “Cuadernillo de Entrenamiento y Optimización de Clasificadores IMU,” Google Colab, 2026. [En línea]. Disponible: <https://colab.research.google.com/drive/1bQH-Nn6pYv07GIVpCT0562uN-BHrUjrm?usp=sharing>